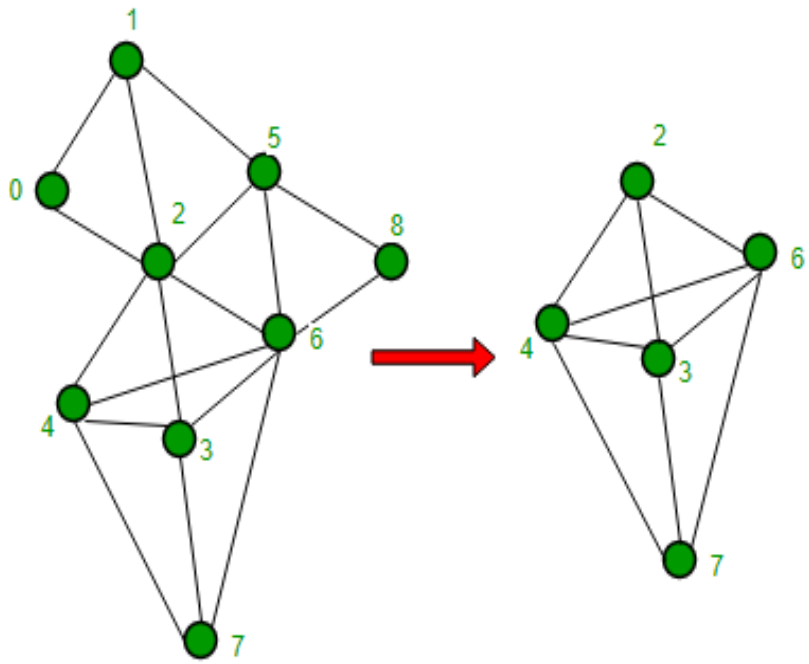
Exercise#1:

Given a graph G and an integer K, K-cores of the graph are connected components that are left after all vertices of degree less than k have been removed.



3-Cores

Input Graph

Example 1 :	
Input	Adjacency list representation of graph shown in the left diagram above. K = 3
Output	{2: [3,4,6], 3: [2,4,6,7], 4: [2,3,6,7], 6: [2,3,4,7], 7: [3,4,6] }

First, we will create an adjacency list.

```
a = {
   0 : [1, 2],
   1 : [0, 2, 5],
   2 : [3, 4, 6],
   3 : [2, 4, 6, 7],
   4 : [2, 3, 6, 7],
   5 : [1, 2, 6, 8],
   6 : [2, 3, 4, 7],
   7 : [3, 4, 6],
   8 : [5, 6]
   }
```

The standard algorithm to find a k-core graph is to remove all the vertices that have degree less than- 'K' from the input graph. We must be careful that removing a vertex reduces the degree of all the vertices adjacent to it, hence the degree of adjacent vertices can also drop below 'K'. And thus, we may have to remove those vertices also. This process may/may not go until there are no vertices left in the graph.

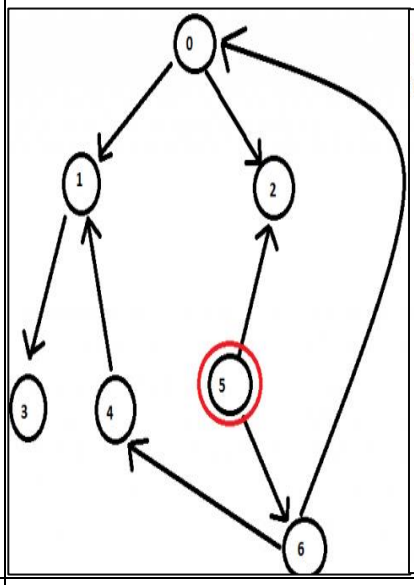
To implement above algorithm, we do a modified DFS on the input graph and delete all the vertices having degree less than 'K', then update degrees of all the adjacent vertices, and if their degree falls below 'K' we will delete them too.

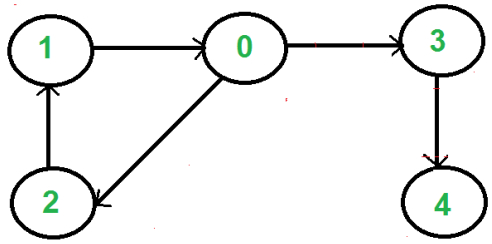
To implement using python, one approach could be:

- **Create an empty list for storing nodes to be removed.**
- **We will run the loop from 0 to size of Graph Nodes.**
    - **Each time, we will loop over the graph nodes & append all the nodes to be removed in the removal nodes list.**
    - **Remove the nodes in removal nodes list from the graph & update the graph.**
- **Return the final graph.**

Exercise # 2

Find a Mother vertex in a graph. Mother vertex in a graph G = (V, E) is a vertex v such that all other vertices in G can be reached by a path from v.

Example 1:	
Input	Adjacency list of the graph shown below  In this graph the mother vertex is- '5'(circled red) as we can reach any node from - '5' through a directed path *To reach 0-* *5->6->0* *To reach 1-* *5->6->0->1* *To reach 2-* *5->2* *To reach 3-* *5->6->0->1->3* *To reach 4-* *5->6->4* *To reach 6-* *5->6*
Output	[5]

Example 2:	
Input	Adjacency list representation of a graph shown below  Mother Vertices : 0, 1 and 2
Output	[0, 1, 2]

How to find mother vertex?

- ***Case 1: Undirected Connected Graph:*** **In this case, all the vertices are mother vertices as we can reach to all the other nodes in the graph.**
- ***Case 2: Undirected/Directed Disconnected Graph*****: In this case, there is no mother vertices as we cannot reach to all the other nodes in the graph.**
- ***Case 3: Directed Connected Graph*****: In this case, we have to find a vertex v in the graph such that we can reach to all the other nodes in the graph through a directed path.**

A trivial approach will be to perform a DFS/BFS on all the vertices and find whether we can reach all the vertices from that vertex.

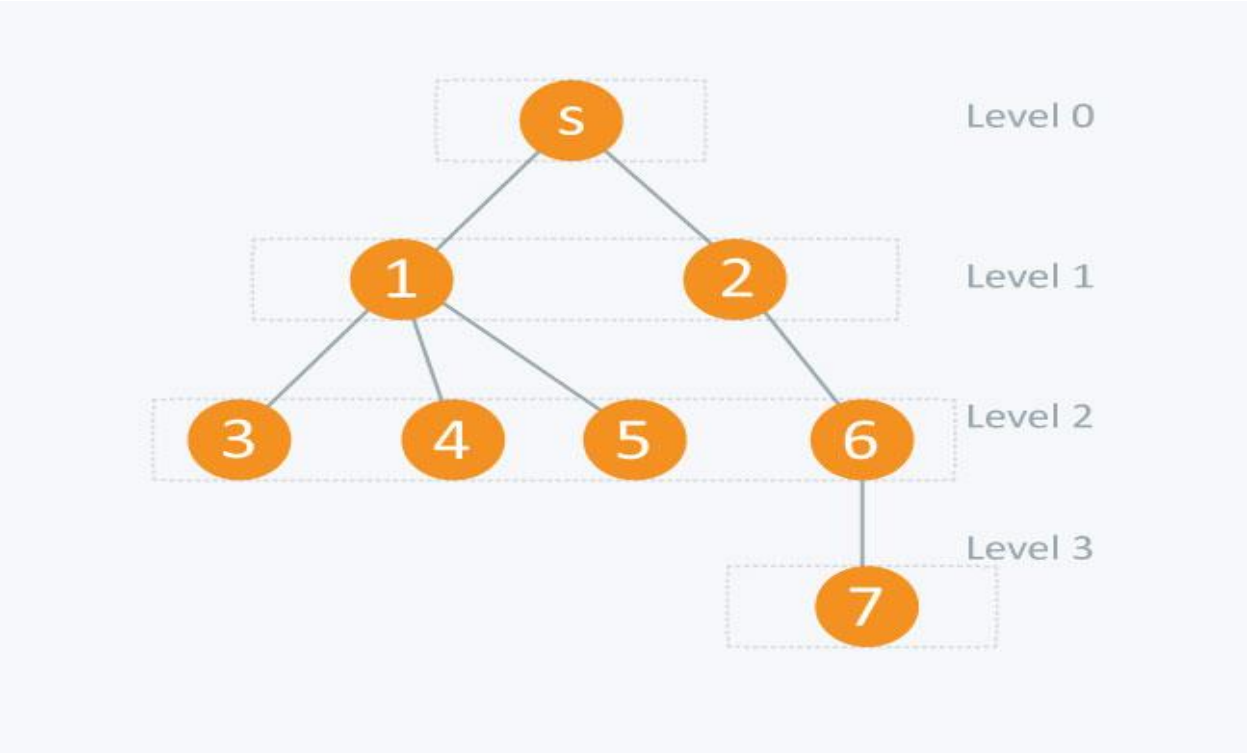
One of the approach is:

- **Loop over the list of nodes / keys in your graph.**
- **For each node/key in your graph,**
    - **Keep that node as starting vertex.**
    - **Perform DFS on it.**
      After performing DFS algorithm, if length of visited list is equal to the size of nodes list, we have found a mother vertex! ☺
- **Return the list of all mother vertices found.**

Exercise#3: Level of each node in the graph

How to determine the level of each node in the given graph?

As you know in BFS, you traverse level wise. You can also use BFS to determine the level of each node.



As you know in BFS, you traverse level wise. You can also use BFS to determine the level of each node.

While you visit each node, the level of that node is set with an increment in the level of its parent node. This is how the level of each node is determined.

```
node                        level [node]
  s (source node)                0
  1                              1
  2                              1
  3                              2
  4                              2
  5                              2
  6                              2
  7                              3
```

https://stackoverflow.com/questions/31247634/how-to-keep-track-of-depth-in-breadth-first-search

Maintain a queue storing the depth of the corresponding node in BFS queue. Sample code for your information:

```
queue bfsQueue, depthQueue;
bfsQueue.push(firstNode);
depthQueue.push(0);
while (!bfsQueue.empty()) {
   f = bfsQueue.front();
   depth = depthQueue.front();
   bfsQueue.pop(), depthQueue.pop();
   for (every node adjacent to f) {
      bfsQueue.push(node), depthQueue.push(depth+1);
   }
}
```

Instead of maintaining two separate queues, a single queue can also be used in which you will push a tuple containing the node itself and its level.

Exercise# 3a:

Write a function get_node_level(G, node) that takes a graph and node as input and returns its level.

Example 1 :	
Input	Adjacency list representation of graph shown above. node = 2
Output	1

Example 2 :	
Input	Adjacency list representation of graph shown above. node = 6
Output	2

We can create a single queue in which you will push a tuple containing the node itself and its level.

We will first create the Adjacency List representation of the Graph above:

```
Graph = {
   "s" : [1, 2],
    1  : [3, 4, 5],
    2  : [6],
    3  : [],
    4  : [],
    5  : [],
    6  : [7],
    7  : []
}
```

One of the approaches could be:
We will have a Queue and a Visited List.
We will put the starting vertex in the Queue by creating a tuple containing the starting vertex and its level (ZERO).
Queue = [(StartingVertex, 0)]
Visited = []

Now until our queue is not empty, we will:

- **De-Queue an item from the queue and store it in a variable i.e. vertex. The vertex will be a tuple in a form of (node, level).**
- **We will check if vertex's node i.e. vertex[0] is the required node. The vertex is in a form of a tuple in a form of (node, level). If it is the required node, return its level i.e. vertex[1].**
- **We will check that is vertex in our visited list i.e. has vertex already been visited. If it is not in the visited list, append the vertex in the visited list. It is necessary to append only those vertices that are not visited because if we visit the same node more than once, we may end up in an infinite loop.**
- **Next, we will loop over the list of neighbors of Graph[vertex[0]]. Since the vertex will be a tuple in a form of (node, level), we need to loop over the neighbors of the node.**
  For each neighbor, if the neighbor not in visited list, push that neighbor in the queue by creating a tuple containing the neighbour and its level (vertex/parent level + 1). i.e. Queue.append((neighbour, vertex[1] + 1)).
- **After the queue is empty, we have completed our BFS traversal. And our Visited list contains all the visited nodes in the form of tuples i.e. (node, level).**
- **Return -1 if the node is not found in the graph.**

```
node                        level [node]
  s (source node)                0
  1                              1
  2                              1
  3                              2
  4                              2
  5                              2
  6                              2
  7                              3
```

Exercise#4: Course Schedule

Bob loves foreign languages and wants to plan his course schedule for the following years.

He is interested in the following nine language courses: LA15, LA16, LA22, LA31, LA32, LA126, LA127, LA141, and LA169.

The course prerequisites are:

- LA15: (none)
- LA16: LA15
- LA22: (none)
- LA31: LA15
- LA32: LA16, LA31
- LA126: LA22, LA32
- LA127: LA16
- LA141: LA22, LA16
- LA169: LA32

In what order can Bob take these courses, respecting the prerequisites?

Example 1 :	
Input	List of Courses & Course Pre-Requisites
Output	There can be different outputs/course plan such as: • LA15, LA22, LA16, L31, L32, LA169, LA126, LA127, LA141 • LA15, LA22, LA16, LA31, LA32, LA127, LA141, LA126, LA169 • LA15, LA16, LA22, LA31, LA32, LA126, LA127, LA141, LA169

We will first create Graph Adjacency List.

```
CourseSchedule =
{
   "LA15" : [],
   "LA16" : ["LA15"],
   "LA22" : [],
   "LA31" : ["LA15"],
   "LA32" : ["LA16", "LA31"],
   "LA126": ["LA22", "LA32"],
   "LA127": ["LA16"],
   "LA141": ["LA22", "LA16"],
   "LA169": ["LA32"]
}
```

Any approach could be used and there can be different outputs. The main constraint that needs to be taken care of is that pre-requisite requirements have been taken into consideration.

One of the approaches could be:

- **We will need a function getLowestDegreeKeys(Graph) that takes a graph as parameter and returns the list of nodes having the lowest degrees in the graph.**
- **We will need a function remove_node (G, node) that takes a graph and node as parameter and remove the node from the graph.**
- **In our createCourseSchedule(Graph) function:**
    - **We will create an empty course_plan list.**
    - **While our Graph length is not ZERO, we will:**
        - **We will call the getLowestDegreeKeys(Graph) to get the list of nodes with lowest degree. Store the list returned by the function in a variable: ListOfNodes.**
        - **We will loop over the list of ListOfNodes:**
            - **We will append that node in the course_plan.**
            - **We will remove that node from the graph using remove_node function.**
    - **When our Graph length is ZERO, our course_plan list is ready. ☺ Return the course_plan list.**

Exercise#4: Course Schedule

Bob loves foreign languages and wants to plan his course schedule for the following years.

He is interested in the following nine language courses: LA15, LA16, LA22, LA31, LA32, LA126, LA127, LA141, and LA169.

The course prerequisites are:

- LA15: (none)
- LA16: LA15
- LA22: (none)
- LA31: LA15
- LA32: LA16, LA31
- LA126: LA22, LA32
- LA127: LA16
- LA141: LA22, LA16
- LA169: LA32

In what order can Bob take these courses, respecting the prerequisites?

Example 1 :	
Input	List of Courses & Course Pre-Requisites
Output	There can be different outputs/course plan such as: • LA15, LA22, LA16, L31, L32, LA169, LA126, LA127, LA141 • LA15, LA22, LA16, LA31, LA32, LA127, LA141, LA126, LA169 • LA15, LA16, LA22, LA31, LA32, LA126, LA127, LA141, LA169

We will first create Graph Adjacency List.

```
CourseSchedule =
{
   "LA15" : [],
   "LA16" : ["LA15"],
   "LA22" : [],
   "LA31" : ["LA15"],
   "LA32" : ["LA16", "LA31"],
   "LA126": ["LA22", "LA32"],
   "LA127": ["LA16"],
   "LA141": ["LA22", "LA16"],
   "LA169": ["LA32"]
}
```

Any approach could be used and there can be different outputs. The main constraint that needs to be taken care of is that pre-requisite requirements have been taken into consideration.

One of the approaches could be:

- **We will need a function getLowestDegreeKeys(Graph) that takes a graph as parameter and returns the list of nodes having the lowest degrees in the graph.**
- **We will need a function remove_node (G, node) that takes a graph and node as parameter and remove the node from the graph.**
- **In our createCourseSchedule(Graph) function:**
    - **We will create an empty course_plan list.**
    - **While our Graph length is not ZERO, we will:**
        - **We will call the getLowestDegreeKeys(Graph) to get the list of nodes with lowest degree. Store the list returned by the function in a variable: ListOfNodes.**
        - **We will loop over the list of ListOfNodes:**
            - **We will append that node in the course_plan.**
            - **We will remove that node from the graph using remove_node function.**
    - **When our Graph length is ZERO, our course_plan list is ready. ☺ Return the course_plan list.**